XJTLU

XJTLU Entrepreneur College (Taicang)

Module Code and Title	ENT208TC Industry Readiness
School Title	XJTLU Entrepreneur College (Taicang)
Assignment Title	Technical Documentation
Submission Deadline	**Friday 15 May, 2026 before 23:59 (Beijing Time, GMT+8)**
Session and Group	[Session 1 — Group 27]
Student IDs	[2362452, 2360861, 2361326, 2361210, 2361098, 2364028, 2362478 , 2361871]
Final Word Count	1826
If you agree to let the university use your work anonymously for teaching and learning purposes, please type **"yes"** here.	**yes**

I certify that I have read and understood the University's Policy for dealing with Plagiarism, Collusion, and the Fabrication of Data (available on Learning Mall Online). With reference to this policy, I certify that:

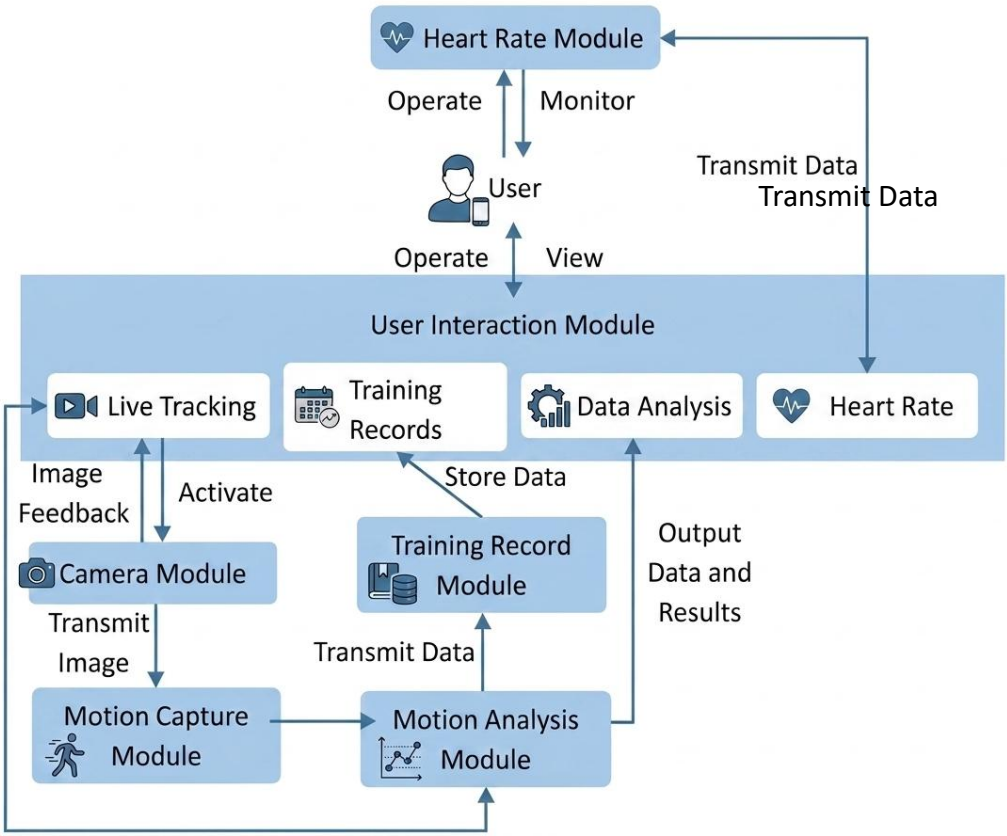
- My work does not contain any instances of plagiarism and/or collusion. My work does not contain any fabricated data.

By uploading my assignment onto Learning Mall Online, I formally declare that all of the above information is true to the best of my knowledge and belief.

Scoring — For Tutor Use							
Student ID	**D**						
Stage of Marking	**Marker Code**	**Learning Outcomes Achieved (F/P/M/D)**					**Final Score**
		A	**B**	**C**	**D**	**E**	
1st Marker — red pen							
Moderation — green pen	IM Initials	The original mark has been accepted by the moderator: Y / N					Y / N
2nd Marker if needed							

Xi'an Jiaotong-Liverpool University
西交利物浦大学

Section 1 — System Architecture



System Architecture Diagram

Component Overview

1. Camera Module:

   This module captures raw video data in real-time through the mobile device's camera while the user exercises. It serves as the physical foundation for system perception, responsible for capturing the video stream to ensure subsequent processing can detect movement changes.

2. Motion Capture Module:

   This module employs advanced computer vision algorithms to accurately extract key skeletal coordinates and joint movement trajectories from the raw video. Its core function is to convert unstructured video frames into machine-readable structured data — joint angles, bar position, and displacement.

3. Motion Analysis Module:

Acting as the "brain" of the system, this module utilizes AI models to evaluate captured motion data in real-time and compares it against standard biomechanical models. It accurately identifies whether movements are standard and instantly generates corrective suggestions alongside multi-dimensional performance charts. T

4. Training Record Module:

This module handles persistent data storage, securely saving historical information such as raw parameters of each session, analysis reports, and user heart rates in the database. It allows users to review past training performances and observe their progress trends based on long-term data.

5. Heart Rate Monitoring Module:

Connecting to a companion wristband via Bluetooth, this module synchronizes users' physiological metrics, such as heart rate, in real-time. It integrates internal physiological loads with external motion data to provide users with a comprehensive health assessment. It correlates physiological load with external motion data for a unified training profile.

6. User Interaction Module:

This module is the interface connecting the user to the underlying technology, representing the app interface visible on the device. It includes interfaces such as "Live Tracking," "Analysis," and "Records". It renders processed data as visual dashboards, trajectory plots, and skeleton overlays.FitVision's typical user journey follows a closed-loop data flow from physical perception to digital analysis, culminating in result feedback. Below is the data flow logic for a standard fitness scenario (e.g., Live Tracking):

Data Flow

FitVision's typical user journey follows a closed-loop data flow from physical perception to digital analysis, culminating in result feedback. Below is the data flow logic for a standard fitness scenario (e.g., Live Tracking):

1. Initialization & Data Collection:

   The data flow originates in the "User Interaction Module". When the user enters the "Live Tracking" interface and activates the camera, a command is sent to the "Camera Module". The camera then begins capturing raw image sequences, serving as the primary data source for all subsequent processing.

2. Real-time Processing & Feedback:

   The raw video stream generated by the "Camera Module" is pushed to the "Motion Capture Module" in real-time. Here, the first data transformation occurs: computer vision algorithms convert pixels into structured joint coordinates. At this point, some real-time data (e.g., joint angles, movement speed, equipment displacement) flows back to the "User Interaction Module" and is rendered synchronously on the tracking interface, providing instant visual feedback.

3. Deep Analysis & Logic Processing:

   Simultaneously with motion capture, structured motion data is transmitted to the "Motion Analysis Module". AI algorithms logically assess the standard of the movements, converting coordinates into systematic conclusions (e.g., "squat depth insufficient" or "keep back straight"). These results are then packaged into visual charts and targeted text suggestions.

4. Persistence & Storage:

   When the user ends the workout, all analysis conclusions, motion metrics, and physiological data synchronized via Bluetooth from the "Heart Rate Monitoring Module" are sent to the "Training Record Module". The data is stored in a local or cloud database, ensuring the transition from an instantaneous stream to a permanent record.

5. Retrieval & Review:

   The stored data supports multi-dimensional retrieval. Users can view a deep summary of their current workout via the "Analysis" interface or retrieve historical data through the "Records" interface. This reverse data flow completes the closed loop from training to review, allowing users to compare outcomes across different timeframes.

Section 2 — Technology Justification

Technology	What we chose	Alternatives considered	Why we chose this
Backend Framework	FastAPI + Uvicorn	Flask, Django	Native WebSocket + async; lighter than Django, no extra libs like Flask needs.
Pose Estimation	YOLOv8n-pose (Ultralytics)	MediaPipe Pose, OpenPose, MoveNet	Real-time CPU inference; easier Python deployment than alternatives.
Real-time Comm	WebSocket (ws://)	SSE, HTTP Polling, gRPC Stream	Bidirectional low-latency; SSE is one-way, gRPC needs browser proxy.
Bluetooth Integration	Web Bluetooth API	Chrome Extension BLE, Native App (React Native/Flutter)	No app install needed; limited to Chrome / Edge.
Frontend Framework	Vanilla JavaScript (No framework)	React, Vue, Svelte	Lightweight static page; no build step, instant deploy.

Technical risks and mitigation

During development, we identified and addressed the following major technical risks:

1. YOLO CPU Inference Latency Risk:

   Running deep learning models on low-end CPUs can cause excessive processing times (>50ms/frame), impacting real-time feedback. Mitigation: We exclusively used the computationally lightweight `yolov8n-pose` (nano version) model and configured parameters to reduce redundant calculations (e.g., `conf=0.3 / iou=0.7`). If the system detects a CUDA environment on the user's device, it automatically enables GPU acceleration to further reduce latency.

2. Network Jitter and WebSocket Disconnection Risk:

   Because video streams and pose data rely heavily on the network, fluctuations can cause data loss. Mitigation: We implemented an automatic reconnection mechanism (up to 5 retries) in the frontend client (`yolo_client.js`) and added ping/pong heartbeat keep-alive logic in the backend. During disconnections, the system automatically falls back to a simulated data source to prevent the application from crashing.

Section 3 — Deployment Guide

Environment requirements

Requirement	Version / Notes
Python	3.11+ recommended (Supports FastAPI / Ultralytics)
pip	24.0+ recommended (Python package manager)
Browser	Chrome 121+ or Edge 121+ (Required for Web Bluetooth API support)
OS & Hardware	Windows 10+ / macOS 12+ / Ubuntu 20.04+, must have an available camera (built-in or USB), and approx. 2GB of disk space

Setup steps

1. Clone the repository and enter directory: git clone <your-repo-url> FitVision && cd FitVision

2. Install dependencies [Windows]: python -m venv venv && .\venv\Scripts\activate && pip install -r requirements.txt

3. Install dependencies [macOS / Linux]: python3 -m venv venv && source venv/bin/activate && pip install -r requirements.txt

4. Run the application (Backend):python pose_server.py

(The terminal displaying "Application startup complete." indicates success, default port 8765).

5. Run the application (Frontend):Keep the backend running. Open a new terminal window in the project root and execute:python -m http.server 3000 --bind 127.0.0.1

6. Verify it is working:Open a browser to `http://localhost:3000/login.html` to register and log in. Click "Connect YOLO" to establish the backend WebSocket connection, then click "Connect Device" to bind the Bluetooth heart rate monitor.

Common issues and solutions

Problem	Likely cause	Solution

ModuleNotFoundError: No module named 'cv2'	The opencv-python library is not installed, or the terminal has not activated the correct Python virtual environment.	Ensure the virtual environment is activated (terminal prefix shows (venv)), then manually install: pip install opencv-python>=4.9.0.
Camera 0 opened but no frame received	The camera is being occupied by another application, or the system has not granted camera privacy permissions to the terminal/Python.	Close applications occupying the camera. Try launching with a different camera index: python pose_server.py --camera 1.

Section 4 — IP Strategy

Novelty analysis

While existing fitness applications primarily offer static video tutorials or post-workout manual data logging, FitVision introduces a multi-modal real-time feedback loop. FitVision's core novelty lies in the simultaneous integration of on-device computer vision kinematic analysis (joint tracking) and Bluetooth Low Energy (BLE) physiological monitoring (heart rate). Unlike comparable products such as Keep or Nike Training Club that provide passive video playback, or Tonal which requires expensive proprietary hardware, FitVision democratizes professional-grade biomechanical feedback by utilizing standard smartphone cameras synchronized with wearable biometrics.

Prior art (2–3 examples)

Existing product	What it does	How your solution differs
Mainstream Fitness Apps (e.g., Keep, NTC)	Provides pre-recorded video tutorials, structured training plans, and manual data logging.	FitVision shifts from passive viewing to active monitoring. It uses the camera for AI-driven real-time motion correction rather than relying on self-assessment.
Smart Home Gyms	Offers real-time form	FitVision is a software-only solution

(e.g., Tonal, Mirror)	feedback and tracking but requires highly expensive, wall-mounted proprietary hardware.	using the user's existing smartphone for precise motion tracking, eliminating high hardware barriers.
General Pose Estimation Algorithms (e.g., MediaPipe)	Open-source frameworks providing basic human joint coordinates and skeletal mapping from video inputs.	FitVision builds a specific application layer over raw pose estimation, combining it with proprietary fitness logic, tailored advice, and synchronized heart rate data.

Patent decision

No, we do not plan to seek a patent for FitVision. The core technology heavily utilizes existing open-source frameworks (e.g., computer vision libraries) and standard hardware (smartphone cameras, Bluetooth bands), making it exceedingly difficult to prove the strict "non-obviousness" required for software patents. Furthermore, the time and financial costs of the patent application process are prohibitive for a startup in the fast-moving mobile app market. Finally, enforcing a software patent against competitors who might slightly alter the UI or algorithms is highly impractical and would drain our resources.

Alternative protection

Based on the decision not to patent, we will rely on two alternative strategies:

1. Trade Secrets (Data Moat): The specific parameters, thresholds, and proprietary datasets our AI uses to evaluate "good" versus "bad" form will be kept strictly confidential. As more users utilize FitVision, our machine learning models will become increasingly accurate, creating a "data moat" that is highly difficult for competitors to replicate.

2. First-Mover Advantage & Brand Loyalty: By rapidly deploying the application to the market and continuously updating features based on user feedback, we can capture the target audience early. Building a strong community and seamless user experience will create high switching costs, protecting our competitive advantage even if imitators emerge.

Section 5 — Limitations & Future Work

Known limitations

Limitation	Why it exists	Impact on users
Lighting sensitivity	YOLO keypoint confidence drops in dim or cluttered scenes.	Inaccurate tracking or skeleton loss in poor environments.
Single-view angle constraint	Monocular 2D camera; no depth. Occluded joints yield no angle data.	Must position camera at a clear side view.
4-exercise library	Each exercise needs manual config: angle ranges, trajectory, thresholds.	Only squat, bench, deadlift, OHP are supported.
BLE browser lock	Web Bluetooth API only works in Chrome / Edge.	Firefox/Safari users get no heart-rate data.

Future work

Feature or improvement planned: [Social & Gamification Module] Introduce a "virtual gym" and leaderboard feature, allowing users to share training records and remotely challenge friends. [Expand Library & Calibration] Introduce a semi-supervised learning pipeline to quickly expand supported exercises, and add an "Environment Pre-check Wizard" to assess lighting/backgrounds before recording.

Technical debt to address: [Model Quantization & Energy Optimization] Running uncompressed AI models locally causes high energy consumption. Future priorities include model quantization and network pruning to drastically compress the model size without sacrificing accuracy, thereby reducing CPU/GPU load and improving compatibility with lower-end phones.

Open research question: [Monocular Occlusion Prediction & 3D Reconstruction] How can we use purely 2D smartphone cameras during complex equipment training (where the body is obscured) to accurately predict and reconstruct obscured 3D joint trajectories using time-series deep learning models (like Transformers or RNNs)?

Reference

Chart.js Contributors. (n.d.). Chart.js [Computer software] https://www.chartjs.org/docs/latest/.

DeepSeek. (n.d.). DeepSeek API documentation. https://api-docs.deepseek.com

Fette, I., & Melnikov, A. (2011). The WebSocket protocol (RFC No. 6455). Internet Engineering Task Force. https://doi.org/10.17487/RFC6455

Jocher, G., Chaurasia, A., & Qiu, J. (2023). Ultralytics YOLOv8 [Computer software]. https://github.com/ultralytics/ultralytics

Lin, T.-Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., Dollár, P., & Zitnick, C. L. (2014). Microsoft COCO: Common objects in context. arXiv. https://arxiv.org/abs/1405.0312

Mozilla. (n.d.). Canvas API. MDN Web Docs. https://developer.mozilla.org/en-US/docs/Web/API/Canvas_API

Mozilla. (n.d.). Web Bluetooth API. MDN Web Docs. https://developer.mozilla.org/en-US/docs/Web/API/Web_Bluetooth_API

OpenCV. (n.d.). OpenCV documentation [Computer software library]. https://docs.opencv.org

Ramírez, S. (n.d.). FastAPI [Computer software]. https://fastapi.tiangolo.com

Web Bluetooth Community Group. (n.d.). Web Bluetooth API. W3C Community Group Draft Report. https://webbluetoothcg.github.io/web-bluetooth/